Prompt Log

Operating Systems Final Project – AI-Assisted Linux Kernel Development

Student Name: 許倬允

Student ID: 113502009

Kernel Version: Linux 6.6.30

Environment: Windows + WSL2 Ubuntu + QEMU

Purpose: Document AI-assisted development process for the Operating Systems final project

This prompt log records how AI tools were used as development assistance. All kernel modifications, compilation, QEMU testing, debugging, and result verification were performed manually by the student. AI responses were used for guidance, explanation, debugging support, and report organization.

Entry 1: Understanding the project requirements

Date: 2026-05-27

Prompt: 請幫我看 Operating Systems Final Project 的作業要求，並給我詳細步驟。

AI Response Summary: AI 協助整理作業要求，將專題拆成 Task 1 到 Task 5，包含 QEMU 開機、自編 Linux kernel、新增 system call、kernel modification、benchmark 與 AI prompt log。

How I Used It: 我根據整理後的任務順序，先完成 Task 1 的 QEMU boot，再進行 Task 2 和 Task 3 的 system call 實作。

How I Verified It: 我確認作業 PDF 中的要求與 AI 整理內容一致，並依照任務順序實際操作。

Entry 2: Windows host environment setup

Date: 2026-05-27

Prompt: 我的主機是 Windows，要怎麼完成這份 Linux kernel project？

AI Response Summary: AI 建議使用 Windows + WSL2 Ubuntu + QEMU 的工作流程，並說明在 WSL2 中安裝 kernel build tools、QEMU、BusyBox 與相關套件。

How I Used It: 我使用 WSL2 Ubuntu 作為主要開發環境，在其中下載、設定與編譯 Linux kernel，並建立 BusyBox root filesystem。

How I Verified It: 我在 WSL2 中成功執行 kernel build 指令，並使用 QEMU 載入 bzImage 與 initramfs。

Entry 3: Task 1 QEMU boot validation

Date: 2026-05-27

Prompt: 這樣 Task 1 有成功嗎？

AI Response Summary: AI 說明若 QEMU 顯示 Run /init as init process、Boot successful、BusyBox shell prompt，代表自編 kernel 已成功 boot。

How I Used It: 我使用 QEMU 啟動自編譯 kernel，並確認系統進入 BusyBox shell。

How I Verified It: 我在 QEMU 中看到 BusyBox shell，並可執行 uname、mount、dmesg 等指令，確認 Task 1 成功。

Entry 4: Task 2 system call implementation location

Date: 2026-05-27

Prompt: 我把 SYSCALL_DEFINE1(mycall, int, x) 貼到 BusyBox shell，出現 syntax error，該怎麼辦？

AI Response Summary: AI 說明 C kernel source code 不能貼到 QEMU BusyBox shell，必須在 WSL2 中修改 Linux kernel source，例如 kernel/sys.c、include/linux/syscalls.h 與 syscall_64.tbl。

How I Used It: 我改在 WSL2 的 kernel source tree 中新增 mycall，而不是在 QEMU shell 中輸入 C 程式碼。

How I Verified It: 我重新編譯 modified kernel，並將 user-space 測試程式放入 rootfs 後，在 QEMU 中執行測試。

Entry 5: Task 2 custom syscall mycall

Date: 2026-05-27

Prompt: 請重新列出 Task 2 的完整步驟。

AI Response Summary: AI 提供 Task 2 的完整流程：在 kernel/sys.c 新增 SYSCALL_DEFINE1(mycall, int, x)、在 syscalls.h 宣告 sys_mycall、在 syscall_64.tbl 註冊 syscall number、重新編譯 kernel，並撰寫 test_mycall.c。

How I Used It: 我依照步驟新增 mycall system call，並撰寫 user-space 測試程式呼叫 syscall number。

How I Verified It: 在 modified kernel 中執行 /bin/test_mycall，預期輸出 sys_mycall(7) = 49，用來驗證 syscall 正確回傳平方值。

Entry 6: Static linking issue in BusyBox rootfs

Date: 2026-05-27

Prompt: test_mycall 在 /bin 裡看得到，但執行時顯示 not found，為什麼？

AI Response Summary: AI 解釋在 BusyBox minimal rootfs 中，如果 user-space 程式是 dynamically linked，會因為缺少 dynamic linker 或 glibc 而出現 not found。解法是使用 gcc -static。

How I Used It: 我將 test_mycall、test_get_proc_info 與 bench_matmul 都使用 gcc -static 編譯。

How I Verified It: 使用 file 指令檢查 binary 顯示 statically linked，並在 QEMU 中成功執行。

Entry 7: Task 3 get_proc_info design

Date: 2026-05-27

Prompt: 直接進 Task 3。

AI Response Summary: AI 建議選擇 Direction 3，實作 get_proc_info(pid, &info) system call，用來回傳 process 的 voluntary context switches、involuntary context switches、minor page faults 與 major page faults。

How I Used It: 我將 Task 3 設計成新的 system call，透過 PID 查詢 task_struct 中的統計資訊，並使用 copy_to_user 回傳給 user-space。

How I Verified It: 我撰寫 test_get_proc_info.c，執行後可印出 PID、context switches 與 page faults，確認 syscall 功能。

Entry 8: Task 4 CPU-bound benchmark

Date: 2026-05-27

Prompt: 給我 Task 4 詳細步驟，我要做 CPU-bound 那個。

AI Response Summary: AI 建議使用矩陣乘法作為 CPU-bound benchmark，並提供 bench_matmul.c 的設計方向，包括測量 Execution Time、CPU Time、CPU Utilization 與 Throughput MFLOPS。

How I Used It: 我使用 matrix multiplication benchmark 比較 standard kernel 與 modified kernel 的效能差異。

How I Verified It: benchmark 在兩個 kernel 上各執行至少三次，並記錄輸出的 execution time、CPU utilization 與 throughput 後計算平均。

Entry 9: Standard and modified kernel comparison

Date: 2026-05-27

Prompt: standard kernel 沒有 mycall，modified kernel 有 mycall，要怎麼測試？

AI Response Summary: AI 說明 standard kernel 應作為 baseline，不包含自訂 syscall；modified kernel 則包含 Task 2 和 Task 3 的修改。兩者使用相同 rootfs 與 benchmark 程式進行比較。

How I Used It: 我建立 linux-6.6.30-standard 與 linux-6.6.30-modified 兩份 kernel tree，並分別使用 QEMU 開機測試。

How I Verified It: 我檢查 QEMU 啟動路徑是否指向正確 bzImage，避免誤用 /path/to/... 這種範例路徑。

Entry 10: Final report structure

Date: 2026-05-27

Prompt: 幫我寫一個詳細的 final submission 範本。

AI Response Summary: AI 協助建立 final submission 結構，包含 report、prompt_log、kernel_patch、source_code、screenshots 與 demo script。

How I Used It: 我根據建議整理最終繳交資料夾，將 kernel patch、test programs、benchmark code、report 與 prompt log 分開放置。

How I Verified It: 我檢查 final submission checklist，確保包含 code/patch、test programs、benchmark、PDF report、prompt log 與 demo 資料。

Entry 11: System design and implementation details

Date: 2026-05-27

Prompt: System design 要怎麼寫？Implementation details 要怎麼寫？

AI Response Summary: AI 協助區分 System Design 與 Implementation Details。System Design 描述整體架構；Implementation Details 描述實際修改檔案、system call 實作與測試程式。

How I Used It: 我將報告分成系統設計、實作細節、測試方法、效能分析與 AI-assisted development process。

How I Verified It: 我確認報告內容涵蓋作業要求的 system design、implementation details、testing methodology、performance analysis 與 AI-assisted development process。

Entry 12: Modified files documentation

Date: 2026-05-27

Prompt: 給我有修改過的檔案的路徑和解釋它的功能。

AI Response Summary: AI 幫助整理修改或新增的檔案，包括 kernel/sys.c、include/linux/syscalls.h、syscall_64.tbl、include/uapi/linux/proc_info.h、test_mycall.c、test_get_proc_info.c、bench_matmul.c 與 rootfs/init。

How I Used It: 我將修改檔案與功能說明放入報告的 Implementation Details 或 Appendix。

How I Verified It: 我用檔案路徑檢查 modified kernel tree 與 source_code 資料夾，確認提交檔案與報告描述一致。

Conclusion

AI tools were used to support the development process, especially for understanding Linux kernel development workflow, debugging common errors, organizing benchmark methodology, and preparing the final report structure. However, the actual implementation, compilation, testing, and verification were completed manually. The prompt log demonstrates that AI was used as an assistant while maintaining full understanding and control of the project implementation.